



REDES
CÁTEDRA ÁLVAREZ HAMELIN
CURSO JUEVES

Trabajo Práctico 2

SDN - NAT



25 de junio de 2026

María Florencia Paisan
111679

Tomás Pérez D'Angelo
111834

Juana Segura
111805

Francisco Serrano
111813

Índice

1. Descripción general de la solución	3
2. Diseño de la Tabla NAT	4
2.1. Algoritmo de asignación y resolución de colisiones	4
2.2. Liberación de recursos y consistencia	4
3. Manejo de ARP	5
4. Estrategia de Instalación de Flujos	6
4.1. Ciclo de vida	6
5. Decisiones de diseño	7
6. Problemas encontrados y soluciones implementadas	8
6.1. Compatibilidad con el Protocolo ICMP (Ping)	8
7. Pruebas realizadas	9
7.1. Preparación del entorno y comandos utilizados	9
7.1.1. Parámetros de xterm (Terminales de host)	9
7.1.2. Parámetros de iperf (Generador de tráfico)	9
7.2. Prueba 1: Servidor y un solo cliente (Diferentes tipos de tráfico)	9
7.2.1. Caso A: Tráfico Orientado a Conexión (TCP)	9
7.2.2. Caso B: Tráfico no orientado a conexión (UDP)	12
7.3. Prueba 2: Múltiples clientes simultáneos (conurrencia)	14
7.3.1. Caso A: Conurrencia con TCP	14
7.3.2. Caso B: Conurrencia con UDP	16
7.4. Validación del comportamiento mediante tcpdump	18
7.4.1. Prueba de validación con tráfico HTTP (TCPDump)	18
8. Conclusión	20

1. Descripción general de la solución

En el presente trabajo práctico se implementó un router con funcionalidad de NAT saliente con traducción por puertos (PAT) utilizando el paradigma de Redes Definidas por Software (SDN).

Se partió de una topología emulada en **Mininet** (`topo.py`) en una red privada (con múltiples hosts como **h2** y **h3**) y una red pública (**h1**) conectadas por un switch **OpenFlow**. El controlador, desarrollado en Python sobre POX, asume el rol del plano de control.

2. Diseño de la Tabla NAT

Para el registro, seguimiento y traducción de las conexiones activas, se encapsuló la lógica de estado en la clase `NatManager`. Este módulo actúa como el núcleo del PAT (Port Address Translation), encargándose de asociar los flujos internos con los externos, resolver colisiones en la asignación de puertos y garantizar la consistencia del estado de las conexiones.

Se implementaron dos estructuras de datos principales:

- **Tabla de Traducción NAT (`self.nat_table`):** Es un diccionario cuya clave es una tupla compuesta por el protocolo de transporte, la dirección IP de origen privada y el puerto de origen privado:

$\text{Clave} = \langle \text{nw_proto}, \text{private_ip}, \text{private_port} \rangle$

El valor almacenado es el `public_port` asignado para esa conexión.

La inclusión del protocolo en la clave resulta mandatoria para permitir la coexistencia de flujos TCP y UDP que utilicen idénticos direccionamientos. Además, se optó por omitir la IP pública dentro de la estructura de la tabla dado que el router opera con una única interfaz externa con IP fija (`PUBLIC_IP`), entonces su almacenamiento sería redundante.

- **Registro de puertos en uso (`self.used_ports`):** Es un *set* que almacena tuplas de la forma `(nw_proto, port)`. Esta estructura funciona como un indexador que refleja qué puertos del rango asignable (entre el 49152 y el 65535) se encuentran reservados, agilizando la validación de disponibilidad al establecer nuevos flujos.

2.1. Algoritmo de asignación y resolución de colisiones

Cuando un host de la red privada inicia un flujo hacia la red pública y no cuenta con un mapeo previo en `self.nat_table`, el módulo de NAT ejecuta la asignación:

1. **Intento de conservación de puerto:** Inicialmente, se intenta asignar como puerto público el mismo puerto de origen que solicitó el host privado (`private_port`), siempre que este se encuentre dentro del rango permitido.
2. **Búsqueda secuencial:** Si el puerto base ya se encuentra registrado en `self.used_ports` para ese mismo protocolo, el algoritmo inicia un ciclo iterativo incremental (`port += 1`). Si la búsqueda alcanza el límite superior (`MAX_PORT`), se produce un desborde que reinicia la búsqueda desde el límite inferior (`MIN_PORT`).
3. **Prevención de bucle infinito:** Para prevenir bucles infinitos, el ciclo lleva un contador de intentos limitados a la capacidad máxima del rango disponible. Si no se halla un puerto libre tras recorrer todo el espacio de búsqueda cierta cantidad de veces, el método retorna un valor nulo (`None`) y el paquete es descartado.

2.2. Liberación de recursos y consistencia

Para evitar la fuga de memoria y la retención indefinida de puertos públicos, la tabla NAT se sincroniza con el ciclo de vida de las reglas instaladas en el switch **OpenFlow**.

Al expirar el timer de inactividad (`idle_timeout`) de un flujo en el switch, este remite un evento del tipo `FlowRemoved` hacia el controlador. Al capturar este mensaje, el método `release_port_from_flow` inspecciona los campos del `match` afectado, extrae la tupla de conexión original y elimina la entrada correspondiente en `self.nat_table` junto con su puerto reservado en `self.used_ports`. De este modo, los puertos públicos quedan inmediatamente disponibles para ser reutilizados por nuevas conexiones.

3. Manejo de ARP

Se implementó un mecanismo de resolución dinámica de direcciones físicas en `ArpHandler`, la cual opera en el plano de control e interactúa con el router a través de cuatro mecanismos:

- **Aprendizaje:** El controlador inspecciona todos los paquetes ARP (tanto *Requests* como *Replies*) que pasan por el switch. Independientemente del destino del paquete, se extrae el origen (IP, MAC) y se registra en una estructura de acceso rápido (`self.arp_table`). Esto llena la tabla y minimiza la necesidad de emitir consultas futuras a la red.
- **Intercepción y respuesta local:** El router actúa como frontera para ambas redes (pública y privada). Cuando un host emite un *ARP Request* consultando por la dirección física de alguna de las interfaces del router (cuyas IPs están en `self.our_addresses`), el `ArpHandler` intercepta el mensaje. Inmediatamente, crea un *ARP Reply* a través del método `send_reply` dirigido al solicitante, otorgando la MAC correspondiente.
- **Resolución activa y buffering:** Si llega un paquete IP cuyo destino no posee una entrada en la tabla ARP, el sistema evita descartar el datagrama. En su lugar, invoca el método `resolve_or_queue`, el cual retiene el paquete en un buffer temporal (`self.pending`) indexado por la IP de destino. Simultáneamente, emite un único *ARP Request* en *broadcast* hacia la red. Este diseño previene un *flood* de broadcasts, ya que si llegan múltiples paquetes consecutivos para la misma IP desconocida, se agrupan en la cola esperando la resolución de la primera consulta.
- **Callbacks:** Al recibir el *ARP Reply* esperado por parte de la red, el módulo registra la nueva MAC y procede a desencolar todos los eventos de paquetes que se encontraban en pausa para esa dirección IP específica. Estos paquetes son metidos nuevamente al flujo principal de procesamiento IP ejecutando el *callback* `self.on_resolved` (el cual referencia a `handle_ip` del router), permitiendo que continúen su ciclo normal hacia el proceso de NAT y la instalación de flujos **OpenFlow** en el switch.

4. Estrategia de Instalación de Flujos

Una vez que el **NatManager** establece los parámetros de traducción para una nueva conexión, el controlador delega el reenvío de los paquetes directamente al *switch* mediante la instalación de reglas **OpenFlow** (abstraídas en el módulo `flow_manager.py`).

Por cada nueva sesión detectada en la red, se instalan dos flujos mutuamente dependientes que garantizan una comunicación bidireccional sin requerir nuevas intervenciones del plano de control:

1. Flujo saliente (Outbound):

- **Match:** Intercepta datagramas que ingresan por el puerto de la red privada, filtrando por el protocolo de capa de red (IPv4, `d1_type=0x800`), el protocolo de transporte (TCP/UDP), y la tupla de IP y puerto origen del host local.
- **Action:** Muta el paquete reescribiendo su *header*. Reescribe la dirección MAC origen con la de la interfaz pública del router y la MAC destino con la del próximo salto (resuelta vía ARP). Se enmascara la IP origen con la `PUBLIC_IP` y reemplaza el puerto origen con un puerto público reservado. Finalmente, emite el paquete por la interfaz.

2. Flujo entrante (Inbound):

- **Match:** Captura el tráfico proveniente de la red pública cuyo destino coincida con la `PUBLIC_IP` del router y el puerto de transporte público previamente asignado a esta conexión.
- **Action:** Aplica la traducción inversa. Restaura la IP y el puerto de destino a los valores originales del host en la red privada. Se ajustan las direcciones MAC para direccionar el paquete hacia el host interno correspondiente y se envía el paquete por el puerto local.

4.1. Ciclo de vida

Para garantizar un uso eficiente de la memoria del switch y permitir la rotación de puertos, ambos flujos se instalan con un `idle_timeout` de 10 segundos y el indicador `OFPPF_SEND_FLOW_REM`. Esta flag es crítica, ya que le enseña al switch a notificar al controlador cuando una regla expira por falta de tráfico. Al recibir esta señal, se puede liberar el puerto público asociado.

5. Decisiones de diseño

Con el objetivo de lograr un router SDN eficiente, se tomaron las siguientes decisiones de diseño:

- **Arquitectura modular:** Se optó por una separación de responsabilidades en módulos independientes. **NatManager** centraliza la lógica de traducción (PAT), **ArpHandler** encapsula la gestión de capa 2 y las resoluciones asíncronas, y **FlowManager** actúa como una capa de abstracción para la interacción con el switch **OpenFlow**.
- **Gestión de estado NAT y reutilización de puertos:** Se utilizó un diccionario (`self.nat_table`) para el mapeo rápido de flujos y un set (`self.used_ports`) para mantener un registro de puertos. La elección de incluir el protocolo (TCP/UDP) en la clave de la tabla NAT permite que coexistan conexiones con idénticos puertos origen/destino pero distinto protocolo.
- **Ciclo de vida:** Se decidió utilizar el flag `OFPPF_SEND_FLOW_REM` junto con un `idle_timeout` de 10 segundos. Esta decisión delega la responsabilidad de la liberación de recursos al switch: cuando una regla expira, el evento `FlowRemoved` notifica al controlador para liberar el puerto público, evitando la fuga de memoria en el plano de control y garantizando la disponibilidad inmediata de puertos para nuevas conexiones.
- **Manejo de ARP y queueing:** Para evitar el descarte de paquetes durante la resolución de direcciones físicas, se implementó un sistema de *queueing* (`self.pending`) en **ArpHandler**. Los paquetes IP que esperan una respuesta ARP no se descartan, sino que se encolan y se procesan mediante un *callback* una vez resuelta la MAC. Esto reduce la latencia percibida por la aplicación y evita el *flooding* de consultas ARP en la red.
- **Traducción de Direcciones y Puertos (PAT):** Dado que el enrutador opera con una única interfaz externa con IP constante (`PUBLIC_IP`), se tomó la decisión de omitir la IP pública en la estructura de la tabla de estado, optimizando el uso de memoria en el controlador. La traducción de puertos se realiza mediante una estrategia de asignación incremental que intenta preservar el puerto original del host privado siempre que sea posible, incrementando de forma secuencial solo en caso de colisión.
- **Elección del timeout:** Se definió un `idle_timeout` de 10 segundos para los flujos del NAT. Esta decisión se tomó teniendo en cuenta el beneficio de la rápida liberación de recursos y la carga de procesamiento del controlador. Al utilizar un timeout corto, el componente **NatManager** puede reciclar rápidamente los puertos públicos asignados en la tabla de traducción una vez que termina el tráfico de una conexión activa, previniendo el agotamiento de puertos. Si bien un valor reducido incrementa la probabilidad de que una ráfaga posterior de la misma conexión deba ser procesada nuevamente por el controlador, en el contexto de un entorno emulado en Mininet este valor resultó óptimo para realizar las pruebas y validaciones necesarias.

6. Problemas encontrados y soluciones implementadas

6.1. Compatibilidad con el Protocolo ICMP (Ping)

Problema: Al verificar la conectividad básica con `ping` una vez completado el trabajo, el controlador fallaba al intentar extraer puertos (`srcport`), ya que el protocolo ICMP es de capa 3 y no tiene dichos atributos.

Solución: El alcance de PAT se limitó a TCP y UDP.

7. Pruebas realizadas

7.1. Preparación del entorno y comandos utilizados

Para inicializar el entorno, se ejecutó el controlador POX en una terminal y la topología de Mininet en otra. Posteriormente, se utilizaron terminales independientes (**xterm**) para interactuar de forma aislada con cada host.

A continuación, se detallan las herramientas y los parámetros específicos empleados:

7.1.1. Parámetros de xterm (Terminales de host)

Para abrir las consolas de los hosts con la tipografía y el comportamiento adecuado, se utilizó:

```
mininet>h1 xterm -fa Monospace -fs 11 &
```

7.1.2. Parámetros de iperf (Generador de tráfico)

Se empleó la herramienta **iperf** con los siguientes flags:

Parámetro	Función y Descripción
-s	Configura la herramienta en modo Servidor , poniéndola a la escucha de conexiones.
-c <IP>	Configura la herramienta en modo Cliente , indicando la IP destino del servidor.
-u	Fuerza el uso del protocolo UDP en lugar del protocolo TCP por defecto.
-t 20	Especifica el Tiempo total de la transmisión en segundos (por defecto son 10).

7.2. Prueba 1: Servidor y un solo cliente (Diferentes tipos de tráfico)

El objetivo de este escenario es validar el ciclo de vida completo de un flujo individual, analizando la resolución ARP, la traducción de los encabezados IP/Puerto y la inyección de reglas **OpenFlow** en el switch.

7.2.1. Caso A: Tráfico Orientado a Conexión (TCP)

Se iniciaron las instancias y se realizó la medición mediante los siguientes comandos:

```
1 # En la CLI de mininet (para levantar wireshark de fondo):
2 mininet> h1 wireshark >/dev/null 2>&1 &
3 mininet> h2 wireshark >/dev/null 2>&1 &
4
5 # En la xterm de h1 (Servidor publico):
6 iperf -s
7
8 # En la xterm de h2 (Cliente privado):
9 iperf -c 200.0.0.1 -t 10
```

Mientras corría el tráfico, se consultó la tabla de flujos en una terminal externa:

```
sudo ovs-ofctl dump-flows s1
```

```
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2# iperf -s
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 200.0.0.1 port 5001 connected with 200.0.0.254 port 49152 (icwnd/mss/irtt=14/1448/276)
[ ID] Interval      Transfer           Bandwidth
[ 1] 0.0000-10.0002 sec 20.7 Gbytes      17.7 Gbits/sec
```

Figura 1: Consola iperf en Servidor h1 (TCP).

```
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2# iperf -c 200.0.0.1
Client connecting to 200.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 192.168.1.2 port 42248 connected with 200.0.0.1 port 5001 (icwnd/mss/irtt=14/1448/42855)
[ ID] Interval      Transfer           Bandwidth
[ 1] 0.0000-10.0143 sec 20.7 Gbytes      17.7 Gbits/sec
```

Figura 2: Consola iperf en Cliente h2 (TCP).

Capturing from h1-eth0

No.	Time	Source	Destination	Protocol	Length	Hop Limit	Info
1	0.000000000	00:00:00:aa:aa:aa	Broadcast	ARP	42		who has 200.0.0.1? Tell 200.0.0.254
2	0.000000979	00:00:00:00:00:01	00:00:00:aa:aa:aa	ARP	42		200.0.0.1 is at 00:00:00:00:00:01
3	0.041738459	200.0.0.1	200.0.0.254	TCP	74		49152 → 5001 [SYN] Seq=0 Win=0 MSS=1460 SACK_PERM TSval=297625312 TSecr=0
4	0.041739593	200.0.0.1	200.0.0.254	TCP	74		5001 → 49152 [SYN, ACK] Seq=0 Ack=1 Win=0 MSS=1460 SACK_PERM TSval=2168767176 TSecr=0
5	0.042023314	200.0.0.1	200.0.0.1	TCP	66		49152 → 5001 [ACK] Seq=1 Ack=1 Win=0 Len=0 TSval=2976253123 TSecr=2168767176
6	0.042243581	200.0.0.1	200.0.0.1	TCP	126		49152 → 5001 [PSH, ACK] Seq=1 Ack=1 Win=42496 Len=60 TSval=2976253123 TSecr=2168767176
7	0.042248861	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=1 Ack=61 Win=43520 Len=0 TSval=2168767176 TSecr=2976253123
8	0.042271222	200.0.0.1	200.0.0.1	TCP	7306		49152 → 5001 [PSH, ACK] Seq=61 Ack=1 Win=42496 Len=7240 TSval=2976253123 TSecr=2168767176
9	0.042274959	200.0.0.1	200.0.0.254	TCP	94		5001 → 49152 [PSH, ACK] Seq=1 Ack=61 Win=43520 Len=28 TSval=2168767176 TSecr=2976253123
10	0.042278894	200.0.0.1	200.0.0.1	TCP	7306		49152 → 5001 [PSH, ACK] Seq=7301 Ack=1 Win=42496 Len=7240 TSval=2976253123 TSecr=2168767176
11	0.042295777	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=14541 Win=55808 Len=0 TSval=2168767176 TSecr=2976253123
12	0.042309087	200.0.0.1	200.0.0.1	TCP	66		49152 → 5001 [ACK] Seq=14541 Ack=29 Win=0 Len=0 TSval=2976253123 TSecr=2168767176
13	0.042322296	200.0.0.1	200.0.0.1	TCP	14546		49152 → 5001 [PSH, ACK] Seq=14541 Ack=29 Win=42496 Len=14480 TSval=2976253123 TSecr=2168767176
14	0.042329018	200.0.0.1	200.0.0.1	TCP	14546		49152 → 5001 [PSH, ACK] Seq=29021 Ack=29 Win=42496 Len=14480 TSval=2976253123 TSecr=2168767176
15	0.042526202	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=43501 Win=55808 Len=0 TSval=2168767177 TSecr=2976253123
16	0.042555196	200.0.0.1	200.0.0.1	TCP	29026		49152 → 5001 [PSH, ACK] Seq=43501 Ack=29 Win=42496 Len=28966 TSval=2976253124 TSecr=2168767177
17	0.042559945	200.0.0.1	200.0.0.1	TCP	26014		[TCP Window Full] 49152 → 5001 [PSH, ACK] Seq=72401 Ack=29 Win=42496 Len=20848 TSval=2976253124 TSecr=2168767177
18	0.042571205	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=99309 Win=55808 Len=0 TSval=2168767177 TSecr=2976253124
19	0.042606480	200.0.0.1	200.0.0.1	TCP	7978		49152 → 5001 [PSH, ACK] Seq=99309 Ack=29 Win=42496 Len=7904 TSval=2976253124 TSecr=2168767177
20	0.042611169	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=107213 Win=69632 Len=0 TSval=2168767177 TSecr=2976253124
21	0.042624854	200.0.0.1	200.0.0.1	TCP	47858		49152 → 5001 [PSH, ACK] Seq=107213 Ack=29 Win=42496 Len=47784 TSval=2976253124 TSecr=2168767177
22	0.042638585	200.0.0.1	200.0.0.1	TCP	21914		[TCP Window Full] 49152 → 5001 [PSH, ACK] Seq=154997 Ack=29 Win=42496 Len=21848 TSval=2976253124 TSecr=2168767177
23	0.042643411	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=154997 Win=69632 Len=0 TSval=2168767177 TSecr=2976253124
24	0.042659387	200.0.0.1	200.0.0.1	TCP	47858		[TCP Window Full] 49152 → 5001 [PSH, ACK] Seq=176245 Ack=29 Win=42496 Len=47784 TSval=2976253124 TSecr=2168767177
25	0.042670324	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=224029 Win=69632 Len=0 TSval=2168767177 TSecr=2976253124
26	0.042705273	200.0.0.1	200.0.0.1	TCP	65226		49152 → 5001 [PSH, ACK] Seq=224029 Ack=29 Win=42496 Len=65166 TSval=2976253124 TSecr=2168767177
27	0.042712197	200.0.0.1	200.0.0.1	TCP	4358		[TCP Window Full] 49152 → 5001 [PSH, ACK] Seq=269705 Ack=29 Win=42496 Len=4728 TSval=2976253124 TSecr=2168767177
28	0.042723206	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=294261 Win=78336 Len=0 TSval=2168767177 TSecr=2976253124
29	0.042747448	200.0.0.1	200.0.0.1	TCP	65226		49152 → 5001 [PSH, ACK] Seq=294261 Ack=29 Win=42496 Len=65166 TSval=2976253124 TSecr=2168767177
30	0.042751137	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=359421 Win=208896 Len=0 TSval=2168767177 TSecr=2976253124
31	0.042759403	200.0.0.1	200.0.0.1	TCP	13242		49152 → 5001 [PSH, ACK] Seq=359421 Ack=29 Win=42496 Len=13176 TSval=2976253124 TSecr=2168767177
32	0.042764191	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=372597 Win=235988 Len=0 TSval=2168767177 TSecr=2976253124
33	0.042773418	200.0.0.1	200.0.0.1	TCP	65226		49152 → 5001 [PSH, ACK] Seq=372597 Ack=29 Win=42496 Len=65166 TSval=2976253124 TSecr=2168767177
34	0.042779470	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=438829 Win=264704 Len=0 TSval=2168767177 TSecr=2976253124
35	0.042785781	200.0.0.1	200.0.0.1	TCP	56138		49152 → 5001 [PSH, ACK] Seq=437757 Ack=29 Win=42496 Len=56072 TSval=2976253124 TSecr=2168767177
36	0.042793736	200.0.0.1	200.0.0.1	TCP	26138		49152 → 5001 [PSH, ACK] Seq=493829 Ack=29 Win=42496 Len=26064 TSval=2976253124 TSecr=2168767177
37	0.042793475	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=493829 Win=55808 Len=0 TSval=2168767177 TSecr=2976253124
38	0.042807932	200.0.0.1	200.0.0.1	TCP	43586		49152 → 5001 [PSH, ACK] Seq=519893 Ack=29 Win=42496 Len=43448 TSval=2976253124 TSecr=2168767177
39	0.042807341	200.0.0.1	200.0.0.254	TCP	66		5001 → 49152 [ACK] Seq=29 Ack=519893 Win=67264 Len=0 TSval=2168767177 TSecr=2976253124

Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface h1-eth0, id 0
h1-eth0 - live capture in progress

Figura 3: Captura de Wireshark en Servidor h1 (TCP).

Capturing from h2-eth0

No.	Time	Source	Destination	Protocol	Length	Hop Limit	Info
1	0.000000000	192.168.1.2	200.0.0.1	TCP	74		42248 → 5001 [SYN] Seq=0 Win=0 MSS=1460 SACK_PERM TSval=2976253880 TSecr=0
2	0.042837837	200.0.0.1	192.168.1.2	TCP	74		5001 → 42248 [SYN, ACK] Seq=0 Ack=1 Win=0 MSS=1460 SACK_PERM TSval=2168767176 TSecr=0
3	0.042872691	192.168.1.2	200.0.0.1	TCP	66		42248 → 5001 [ACK] Seq=1 Ack=1 Win=0 Len=0 TSval=2168767176 TSecr=2976253123
4	0.043178295	192.168.1.2	200.0.0.1	TCP	126		42248 → 5001 [PSH, ACK] Seq=1 Ack=1 Win=42496 Len=60 TSval=2976253123 TSecr=2168767176
5	0.043192371	200.0.0.1	192.168.1.2	TCP	66		5001 → 42248 [ACK] Seq=1 Ack=61 Win=43520 Len=0 TSval=2168767176 TSecr=2976253123
6	0.043206297	192.168.1.2	200.0.0.1	TCP	7306		42248 → 5001 [PSH, ACK] Seq=61 Ack=1 Win=42496 Len=7240 TSval=2976253123 TSecr=2168767176
7	0.043215143	192.168.1.2	200.0.0.1	TCP	94		5001 → 42248 [PSH, ACK] Seq=1 Ack=61 Win=43520 Len=28 TSval=2168767176 TSecr=2976253123
8	0.043225352	200.0.0.1	192.168.1.2	TCP	66		42248 → 5001 [ACK] Seq=29 Ack=14541 Win=55808 Len=0 TSval=2976253123 TSecr=2168767176
9	0.043239077	192.168.1.2	200.0.0.1	TCP	66		5001 → 42248 [ACK] Seq=14541 Ack=29 Win=0 Len=0 TSval=2976253123 TSecr=2168767176
10	0.043240169	200.0.0.1	192.168.1.2	TCP	66		42248 → 5001 [PSH, ACK] Seq=14541 Ack=29 Win=42496 Len=14480 TSval=2976253123 TSecr=2168767176
11	0.043246311	192.168.1.2	200.0.0.1	TCP	14546		42248 → 5001 [PSH, ACK] Seq=29021 Ack=29 Win=42496 Len=14480 TSval=2976253123 TSecr=2168767176
12	0.043252212	192.168.1.2	200.0.0.1	TCP	14546		42248 → 5001 [PSH, ACK] Seq=29021 Ack=29 Win=42496 Len=14480 TSval=2976253123 TSecr=2168767176
13	0.043473290	200.0.0.1	192.168.1.2	TCP	66		5001 → 42248 [ACK] Seq=29 Ack=43501 Win=55808 Len=0 TSval=2168767177 TSecr=2976253123
14	0.043479842	192.168.1.2	200.0.0.1	TCP	29026		42248 → 5001 [PSH, ACK] Seq=43501 Ack=29 Win=42496 Len=28966 TSval=2976253124 TSecr=2168767177
15	0.043480404	192.168.1.2	200.0.0.1	TCP	26014		[TCP Window Full] 42248 → 5001 [PSH, ACK] Seq=72401 Ack=29 Win=42496 Len=20848 TSval=2976253124 TSecr=2168767177
16	0.043514598	200.0.0.1	192.168.1.2	TCP	66		5001 → 42248 [ACK] Seq=29 Ack=99309 Win=55808 Len=0 TSval=2168767177 TSecr=2976253124
17	0.043526688	192.168.1.2	200.0.0.1	TCP	7978		42248 → 5001 [PSH, ACK] Seq=99309 Ack=29 Win=42496 Len=7904 TSval=2976253124 TSecr=2168767177
18	0.043554359	200.0.0.1	192.168.1.2	TCP	66		5001 → 42248 [ACK] Seq=29 Ack=107213 Win=69632 Len=0 TSval=2168767177 TSecr=2976253124
19	0.043558026	192.168.1.2	200.0.0.1	TCP	47858		42248 → 5001 [PSH, ACK] Seq=107213 Ack=29 Win=42496 Len=47784 TSval=2976253124 TSecr=2168767177
20	0.043571208	192.168.1.2	200.0.0.1	TCP	21914		[TCP Window Full] 42248 → 5001 [PSH, ACK] Seq=154997 Ack=29 Win=42496 Len=21848 TSval=2976253124 TSecr=2168767177
21	0.043580387	200.0.0.1	192.168.1.2	TCP	66		5001 → 42248 [ACK] Seq=29 Ack=154997 Win=69632 Len=0 TSval=2168767177 TSecr=2976253124
22	0.043590100	192.168.1.2	200.0.0.1	TCP	4358		[TCP Window Full] 42248 → 5001 [PSH, ACK] Seq=176245 Ack=29 Win=42496 Len=47784 TSval=2976253124 TSecr=2168767177
23	0.043617185	200.0.0.1	192.168.1.2	TCP	66		5001 → 42248 [ACK] Seq=29 Ack=224029 Win=69632 Len=0 TSval=2168767177 TSecr=2976253124
24	0.043631702	192.168.1.2	200.0.0.1	TCP	65226		42248 → 5001 [PSH, ACK] Seq=224029 Ack=29 Win=42496 Len=65166 TSval=2976253124 TSecr=2168767177
25	0.043659100	200.0.0.1	192.168.1.2	TCP	4358		[TCP Window Full] 42248 → 5001 [PSH, ACK] Seq=269705 Ack=29 Win=42496 Len=4728 TSval=2976253124 TSecr=2168767177
26	0.043669141	200.0.0.1	192.168.1.2	TCP	66		5001 → 42248 [ACK] Seq=29 Ack=294261 Win=78336 Len=0 TSval=2168767177 TSecr=2976253124
27	0.043677657	192.168.1.2	200.0.0.1	TCP	65226		42248 → 5001 [PSH, ACK] Seq=294261 Ack=29 Win=42496 Len=65166 TSval=2976253124 TSecr=2168767177
28	0.043693857	200.0.0.1	192.168.1.2	TCP	66		5001 → 42248 [ACK] Seq=29 Ack=359421 Win=208896 Len=0 TSval=2168767177 TSecr=2976253124
29	0.043696371	192.168.1.2	200.0.0.1	TCP	13242		42248 → 5001 [PSH, ACK] Seq=359421 Ack=29 Win=42496 Len=13176 TSval=2976253124 TSecr=2168767177
30	0.043704196	192.168.1.2	200.0.0.1	TCP	65226		42248 → 5001 [PSH, ACK] Seq=372597 Ack=29 Win=42496 Len=65166 TSval=2976253124 TSecr=2168767177
31	0.043709955	192.168.1.2	200.0.0.1	TCP	66		5001 → 42248 [ACK] Seq=29 Ack=372597 Win=235988 Len=0 TSval=2168767177 TSecr=2976253124
32	0.043716188	192.168.1.2	200.0.0.1	TCP	56138		42248 → 5001 [PSH, ACK] Seq=437757 Ack=29 Win=42496 Len=56072 TSval=2976253124 TSecr=2168767177
33	0.043724082	200.0.0.1	192.168.1.2	TCP	66		5001 → 42248 [ACK] Seq=29 Ack=437757 Win=248832 Len=0 TSval=2168767177 TSecr=2976253124
34	0.043728370	192.168.1.2	200.0.0.1	TCP	26138		42248 → 5001 [PSH, ACK] Seq=493829 Ack=29 Win=42496 Len=26064 TSval=2976253124 TSecr=2168767177
35	0.043737667	200.0.0.1	192.168.1.2	TCP	66		500

```
INFO:logger:ARP aprendido: 192.168.1.2 -> 00:00:00:00:00:02
INFO:logger:ARP Reply: 192.168.1.254 está en 00:00:00:bb:bb:bb -> enviado a 192.168.1.2
INFO:logger:RECIBIDO: 192.168.1.2 -> 200.0.0.1 | MAC: 00:00:00:00:00:02 -> 00:00:00:bb:bb:bb | In Port: 2
INFO:logger:MAC desconocida para 200.0.0.1. Encolando paquete y mandando ARP Request
INFO:logger:ARP Request enviado: ¿quién tiene 200.0.0.1?
INFO:logger:ARP aprendido: 200.0.0.1 -> 00:00:00:00:00:01
INFO:logger:ARP resuelto 200.0.0.1 -> 00:00:00:00:00:01, procesando pendientes
INFO:logger:RECIBIDO: 192.168.1.2 -> 200.0.0.1 | MAC: 00:00:00:00:00:02 -> 00:00:00:bb:bb:bb | In Port: 2
INFO:logger:MATCH: 192.168.1.2 pertenece a la red privada 192.168.1.0/24
INFO:logger:NAT SALIENTE: 192.168.1.2:33640 -> 200.0.0.254:49152
INFO:logger:ENVIANDO: 200.0.0.254 -> 200.0.0.1 | MAC: 00:00:00:aa:aa:aa -> 00:00:00:00:00:01 | Out Port: 1
INFO:logger:ARP aprendido: 200.0.0.1 -> 00:00:00:00:00:01
INFO:logger:ARP Reply: 200.0.0.254 está en 00:00:00:aa:aa:aa -> enviado a 200.0.0.1
INFO:logger:LIMPIEZA NAT: Puerto público 49152 liberado.
```

Figura 5: Captura del output del controlador.

Resultados y Verificaciones en TCP:

- **Salida de iperf:** En la xterm del cliente h2 se observa que inicia una conexión local desde su IP privada 192.168.1.2 y puerto 42240 hacia la IP pública destino 200.0.0.1:5001. Sin embargo, en la xterm del servidor h1, la conexión figura como entrante desde la IP pública del router NAT (200.0.0.254) utilizando el puerto remapeado 49152. Esto demuestra que para el servidor la red privada es completamente invisible.
- **Análisis en Wireshark:** Al inspeccionar las capturas se observa el proceso de traducción:
 - **Captura h2-eth0 (lado privado):** El paquete 1 muestra el *Three-way Handshake* con un segmento SYN saliendo con Src IP: 192.168.1.2 y Src Port: 42240. El paquete 2 responde con el SYN, ACK proveniente de 200.0.0.1 hacia las direcciones privadas originales del cliente.
 - **Captura h1-eth0 (lado público):** Previo al tráfico TCP, el router envía un ARP Request preguntando quién tiene la IP 200.0.0.1. Tras recibir el ARP Reply de h1 indicando su MAC (00:00:00:00:00:01), se observa el segmento SYN (paquete 3), pero con sus headers modificados por el switch: Src IP: 200.0.0.254 y Src Port: 49152.
- **Flujos OpenFlow instalados:** La consulta a s1 mediante OpenvSwitch expone las reglas bidireccionales inyectadas por el controlador con un `idle_timeout=10`:

```
1 # Flujo 1: Trafico saliente (Privado -> Publico)
2 cookie=0x0, duration=1.892s, table=0, n_packets=62281, n_bytes=3032702302,
   idle_timeout=10, tcp, in_port="s1-eth2", nw_src=192.168.1.2, nw_dst=200.0.0.1,
   tp_src=42240, tp_dst=5001
3 actions=mod_dl_src:00:00:00:aa:aa:aa, mod_dl_dst:00:00:00:00:00:01, mod_nw_src
   :200.0.0.254, mod_tp_src:49152, output:"s1-eth1"
4
5 # Flujo 2: Trafico entrante (Publico -> Privado)
6 cookie=0x0, duration=1.851s, table=0, n_packets=40189, n_bytes=2652618,
   idle_timeout=10, tcp, in_port="s1-eth1", nw_src=200.0.0.1, nw_dst=200.0.0.254,
   tp_src=5001, tp_dst=49152
7 actions=mod_dl_src:00:00:00:bb:bb:bb, mod_dl_dst:00:00:00:00:00:02, mod_nw_dst
   :192.168.1.2, mod_tp_dst:42240, output:"s1-eth2"
```

Listing 1: Tabla de flujos OpenFlow activa durante la transmisión TCP

Análisis de los campos de los flujos:

n_packets / n_bytes: Indica la cantidad de paquetes y bytes que matchearon con la regla. El flujo saliente procesó 62281 paquetes de datos, mientras que el entrante procesó 40189.

idle_timeout=10: Temporizador de inactividad. Si no pasan paquetes por esta regla durante 10 segundos, el switch la borra automáticamente. Esto coincide con el log del controlador: LIMPIEZA NAT: Puerto publico 49152 liberado, evitando el agotamiento de puertos.

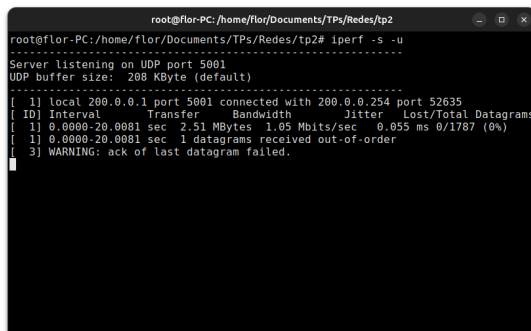
Match fields (tcp, nw_src, nw_dst, tp_src, tp_dst): Campos de capa 3 y 4 indispensables para que el switch identifique este flujo específico.

Actions: En el flujo saliente se realiza la reescritura completa: `mod_nw_src` cambia la IP privada por la pública del router, `mod_tp_src` cambia el puerto de origen a uno controlado (49152), y se modifican las MACs origen/destino para el correcto salto de capa 2 antes de enviarlo por `s1-eth1`. El flujo entrante realiza la operación inversa.

7.2.2. Caso B: Tráfico no orientado a conexión (UDP)

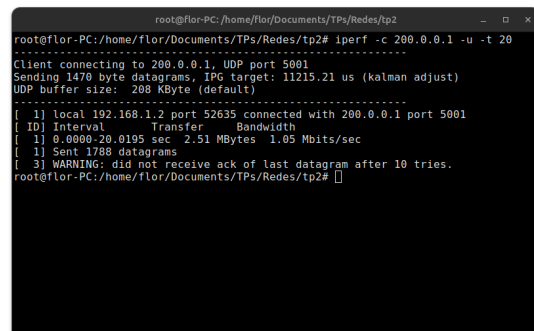
Se repitió la prueba, pero forzando el uso de datagramas UDP para observar el comportamiento frente a ráfagas de tráfico sin *handshake* previo:

```
1 # En la xterm de h1 (Servidor):  
2 iperf -s -u  
3  
4 # En la xterm de h2 (Cliente):  
5 iperf -c 200.0.0.1 -u -t 20
```



```
root@flor-PC: /home/flor/Documents/TPs/Redes/tp2  
root@flor-PC: /home/flor/Documents/TPs/Redes/tp2# iperf -s -u  
-----  
Server listening on UDP port 5001  
UDP buffer size: 208 KByte (default)  
-----  
[ 1] local 200.0.0.1 port 5001 connected with 200.0.0.254 port 52635  
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams  
[ 1] 0.0000-20.0001 sec  2.51 MBytes  1.05 Mbits/sec  0.055 ms  0/1787 (0%)  
[ 1] 0.0000-20.0001 sec  1 datagrams received out-of-order  
[ 3] WARNING: ack of last datagram failed.  
[ ]
```

Figura 6: Consola iperf en Servidor h1 (UDP).



```
root@flor-PC: /home/flor/Documents/TPs/Redes/tp2  
root@flor-PC: /home/flor/Documents/TPs/Redes/tp2# iperf -c 200.0.0.1 -u -t 20  
-----  
Client connecting to 200.0.0.1, UDP port 5001  
Sending 1478 byte datagrams, IPG target: 11215.21 us (kalman adjust)  
UDP buffer size: 208 KByte (default)  
-----  
[ 1] local 192.168.1.2 port 52635 connected with 200.0.0.1 port 5001  
[ ID] Interval      Transfer    Bandwidth  
[ 1] 0.0000-20.0195 sec  2.51 MBytes  1.05 Mbits/sec  
[ 1] Sent 1788 datagrams  
[ 3] WARNING: did not receive ack of last datagram after 10 tries.  
root@flor-PC: /home/flor/Documents/TPs/Redes/tp2# [ ]
```

Figura 7: Consola iperf en Cliente h2 (UDP).

No.	Time	Source	Destination	Protocol	Length	Hop Limit	Info
1	0.000000000	00:00:00:00:00:01	00:00:00:aa:aa:aa	ARP	42		Who has 200.0.0.1? Tell 200.0.0.254
2	0.000012964	00:00:00:00:00:01	00:00:00:aa:aa:aa	ARP	42		200.0.0.1 is at 00:00:00:00:00:01
3	0.010901481	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
4	0.010524025	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
5	0.021639893	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
6	0.032881247	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
7	0.040892753	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
8	0.044871345	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
9	0.055279427	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
10	0.066409513	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
11	0.077711390	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
12	0.088925413	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
13	0.100132873	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
14	0.111346436	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
15	0.122568614	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
16	0.133782997	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
17	0.145007610	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
18	0.156254705	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
19	0.167484948	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
20	0.178699105	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
21	0.189872688	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
22	0.201110115	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
23	0.212320371	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
24	0.223537730	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
25	0.234750711	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
26	0.245942302	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
27	0.257165753	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
28	0.268396617	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
29	0.279599098	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
30	0.290850606	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
31	0.302019781	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
32	0.313243912	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
33	0.324472823	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
34	0.335687327	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
35	0.346895419	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
36	0.358114371	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
37	0.369319961	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
38	0.380521670	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
39	0.391763387	200.0.0.254	200.0.0.1	UDP	1512		52635 → 5901 Len=1470

Figura 8: Captura de Wireshark en Servidor h1 (UDP).

No.	Time	Source	Destination	Protocol	Length	Hop Limit	Info
1	0.000000000	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
2	0.011310784	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
3	0.011333916	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
4	0.022533594	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
5	0.033778665	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
6	0.044974153	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
7	0.056180782	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
8	0.067397199	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
9	0.078612515	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
10	0.089827890	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
11	0.101036963	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
12	0.112256285	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
13	0.123470179	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
14	0.134687528	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
15	0.145910338	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
16	0.157143156	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
17	0.168376168	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
18	0.179573175	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
19	0.190766798	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
20	0.202004058	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
21	0.213217378	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
22	0.224432994	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
23	0.235659744	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
24	0.246884530	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
25	0.258061578	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
26	0.269292983	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
27	0.280499952	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
28	0.291706031	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
29	0.302920495	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
30	0.314141480	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
31	0.325368878	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
32	0.336589772	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
33	0.347791925	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
34	0.359012610	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
35	0.370214250	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
36	0.381422783	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
37	0.392656422	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
38	0.403865596	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470
39	0.415066264	192.168.1.2	200.0.0.1	UDP	1512		52635 → 5901 Len=1470

Figura 9: Captura de Wireshark en Cliente h2 (UDP).

```
INFO:logger:MAC desconocida para 200.0.0.254. Encolando paquete y mandando ARP Request
INFO:logger:RECIBIDO: 200.0.0.1 → 200.0.0.254 | MAC: 00:00:00:00:00:01 → 00:00:00:aa:aa:aa | In Port: 1
INFO:logger:MAC desconocida para 200.0.0.254. Encolando paquete y mandando ARP Request
INFO:logger:RECIBIDO: 200.0.0.1 → 200.0.0.254 | MAC: 00:00:00:00:00:01 → 00:00:00:aa:aa:aa | In Port: 1
INFO:logger:MAC desconocida para 200.0.0.254. Encolando paquete y mandando ARP Request
INFO:logger:RECIBIDO: 200.0.0.1 → 200.0.0.254 | MAC: 00:00:00:00:00:01 → 00:00:00:aa:aa:aa | In Port: 1
INFO:logger:MAC desconocida para 200.0.0.254. Encolando paquete y mandando ARP Request
INFO:logger:RECIBIDO: 200.0.0.1 → 200.0.0.254 | MAC: 00:00:00:00:00:01 → 00:00:00:aa:aa:aa | In Port: 1
INFO:logger:MAC desconocida para 200.0.0.254. Encolando paquete y mandando ARP Request
INFO:logger:RECIBIDO: 200.0.0.1 → 200.0.0.254 | MAC: 00:00:00:00:00:01 → 00:00:00:aa:aa:aa | In Port: 1
INFO:logger:ARP aprendido: 200.0.0.1 -> 00:00:00:00:00:01
INFO:logger:ARP Reply: 200.0.0.254 está en 00:00:00:aa:aa:aa → enviado a 200.0.0.1
INFO:logger:LIMPIEZA NAT: Puerto público 49152 liberado.
```

Figura 10: Captura del output del controlador.

Resultados y Verificaciones en UDP:

- **Salida de iperf UDP:** En las consolas se observa que el cliente h2 inicia la ráfaga desde su IP privada 192.168.1.2 usando el puerto 52635. Se comprueba el correcto funcionamiento del NAT, dado que para h1 los datos ingresaron transparentemente desde la IP pública del router (200.0.0.254:52635). Al finalizar, ambos extremos muestran advertencias (*WARNING: ack of last datagram failed*) debido a la falta de ACK de recibo final.
- **Análisis de comportamiento (ráfaga):** Al ser UDP un protocolo no orientado a conexión con la ausencia de un *Three-way Handshake* y ACKs, analizando los logs del controlador, se observa que la velocidad de inyección de datagramas por parte de la aplicación es mayor al tiempo que le toma al switch resolver las solicitudes ARP e instalar las reglas de reenvío. Como consecuencia de esto, se produce un encolamiento múltiple de paquetes y se disparan múltiples eventos `PacketIn` idénticos y consecutivos (*"MAC desconocida para 200.0.0.254. Encolando paquete..."*) antes de que el switch pueda aplicar las acciones del flujo.
- **Flujos OpenFlow instalados:** La tabla de flujos en s1 demostró la inserción de las dos reglas con la directiva `udp` y el puerto remapeado `tp_src=52635`:

```
1 # Flujo de Salida (Privada -> Publica)
2 cookie=0x0, duration=1.815s, table=0, n_packets=138, n_bytes=208656, idle_timeout
  =10, udp, in_port="s1-eth2", nw_src=192.168.1.2, nw_dst=200.0.0.1, tp_src
  =52635, tp_dst=5001
3 actions=mod_dl_src:00:00:00:aa:aa:aa, mod_dl_dst:00:00:00:00:00:01, mod_nw_src
  :200.0.0.254, mod_tp_src:52635, output:"s1-eth1"
4
5 # Flujo de Entrada (Publica -> Privada)
6 cookie=0x0, duration=1.774s, table=0, n_packets=0, n_bytes=0, idle_timeout=10, udp,
  in_port="s1-eth1", nw_src=200.0.0.1, nw_dst=200.0.0.254, tp_src=5001, tp_dst
  =52635
7 actions=mod_dl_src:00:00:00:bb:bb:bb, mod_dl_dst:00:00:00:00:00:02, mod_nw_dst
  :192.168.1.2, mod_tp_dst:52635, output:"s1-eth2"
```

Listing 2: Tabla de flujos OpenFlow activa durante la transmisión UDP

7.3. Prueba 2: Múltiples clientes simultáneos (conurrencia)

Se ejecutaron transferencias simultáneas desde múltiples hosts privados hacia el único servidor público, forzando la multiplexación de la IP pública del router (200.0.0.254).

7.3.1. Caso A: Conurrencia con TCP

Se configuraron las terminales xterm de los clientes para lanzar peticiones en paralelo:

```
1 # En la xterm de h1 (Servidor):
2 iperf -s
3
4 # En la xterm de h2 (Cliente 1):
5 iperf -c 200.0.0.1 -t 20
6
7 # En la xterm de h3 (Cliente 2):
8 iperf -c 200.0.0.1 -t 20
```

Mientras ambas instancias de `iperf` se ejecutaban en simultáneo, se consultó el estado de la tabla de flujos dinámicos mediante el comando `sudo ovs-ofctl dump-flows s1`.


```
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2# iperf -c 200.0.0.1 -t 20
Client connecting to 200.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 192.168.1.3 port 55092 connected with 200.0.0.1 port 5001 (icwnd/mss/rtt=14/1448/42653)
[ ID] Interval      Transfer     Bandwidth
[ 1] 0.0000-20.0078 sec 29.8 GBytes 12.8 Gbits/sec
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2#
```

Figura 11: Cliente h2: Consola iperf.

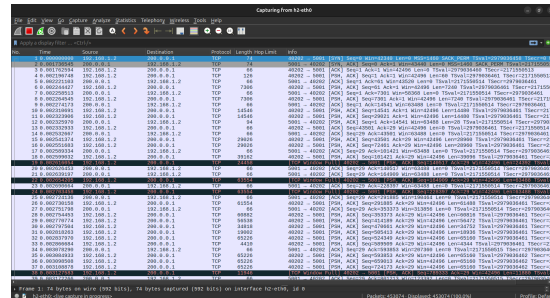


Figura 12: Cliente h2: Captura h2-eth0.

```
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2# iperf -c 200.0.0.1 -t 20
Client connecting to 200.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 192.168.1.2 port 40202 connected with 200.0.0.1 port 5001 (icwnd/mss/rtt=14/1448/1753)
[ ID] Interval      Transfer     Bandwidth
[ 1] 0.0000-20.0145 sec 30.1 GBytes 12.9 Gbits/sec
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2#
```

Figura 13: Cliente h3: Consola iperf.

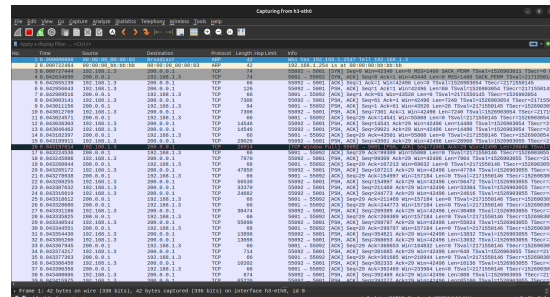


Figura 14: Cliente h3: Captura h3-eth0.

```
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2# iperf -s
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 200.0.0.1 port 5001 connected with 200.0.0.254 port 55092 (icwnd/mss/rtt=14/1448/200)
[ 2] local 200.0.0.1 port 5001 connected with 200.0.0.254 port 49152 (icwnd/mss/rtt=14/1448/200)
[ ID] Interval      Transfer     Bandwidth
[ 1] 0.0000-20.0049 sec 29.8 GBytes 12.8 Gbits/sec
[ 2] 0.0000-20.0036 sec 30.1 GBytes 12.9 Gbits/sec
[SUM] 0.0000-20.3718 sec 59.9 GBytes 25.3 Gbits/sec
```

Figura 15: Servidor h1: Hilos paralelos.

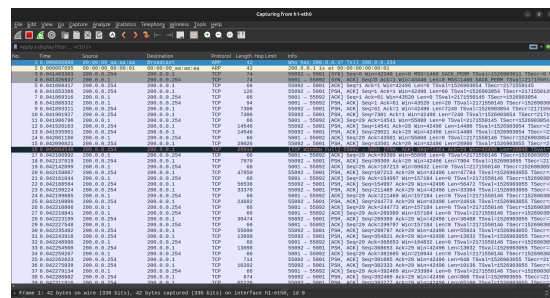


Figura 16: Servidor h1: Captura h1-eth0.

Resultados y Verificaciones en Concurrency TCP:

- **Salida de iperf Concurrente:** Tal como se evidencia en la consola del servidor público h1, ingresan dos flujos de datos dirigidos al puerto de escucha (5001). Sin embargo, ambos flujos figuran como entrantes desde la IP pública del router NAT (200.0.0.254), diferenciados únicamente a nivel de capa 4 por sus puertos remapeados: 55092 y 49152.
- **Análisis de capturas en Wireshark (resolución y multiplexación):** La comparación de las interfaces valida la lógica del router frente al solapamiento:
 - **Segmento de Cliente h_3 :** Envía sus segmentos TCP con el par 192.168.1.3:55092. Al inspeccionar la captura de llegada en el servidor externo h1-eth0, se observa que el

controlador aplicó una preservación de puerto, reescribiendo únicamente la capa de red: 200.0.0.254:55092.

- **Segmento de Cliente h_2 :** Envía sus segmentos TCP utilizando 192.168.1.2:40202. Se ve además que el controlador remapeó este flujo al puerto 49152.
- **Flujos OpenFlow instalados:** La tabla de flujos en s1 demostró la inyección de cuatro reglas simétricas (dos por cada sesión activa):

```
1 # Flujos asociados a h3 (192.168.1.3)
2 cookie=0x0, duration=3.210s, table=0, n_packets=83515, n_bytes=4537956858,
  idle_timeout=10, tcp, in_port="s1-eth3", nw_src=192.168.1.3, nw_dst=200.0.0.1,
  tp_src=55092, tp_dst=5001 actions=mod_dl_src:00:00:00:aa:aa:aa, mod_dl_dst
  :00:00:00:00:00:01, mod_nw_src:200.0.0.254, mod_tp_src:55092, output:"s1-eth1"
3
4 cookie=0x0, duration=3.169s, table=0, n_packets=35868, n_bytes=2367492,
  idle_timeout=10, tcp, in_port="s1-eth1", nw_src=200.0.0.1, nw_dst=200.0.0.254,
  tp_src=5001, tp_dst=55092 actions=mod_dl_src:00:00:00:bb:bb:bb, mod_dl_dst
  :00:00:00:00:00:03, mod_nw_dst:192.168.1.3, mod_tp_dst:55092, output:"s1-eth3"
5
6 # Flujos asociados a h2 (192.168.1.2)
7 cookie=0x0, duration=2.801s, table=0, n_packets=73428, n_bytes=3983071140,
  idle_timeout=10, tcp, in_port="s1-eth2", nw_src=192.168.1.2, nw_dst=200.0.0.1,
  tp_src=40202, tp_dst=5001 actions=mod_dl_src:00:00:00:aa:aa:aa, mod_dl_dst
  :00:00:00:00:00:01, mod_nw_src:200.0.0.254, mod_tp_src:49152, output:"s1-eth1"
8
9 cookie=0x0, duration=2.801s, table=0, n_packets=31004, n_bytes=2046384,
  idle_timeout=10, tcp, in_port="s1-eth1", nw_src=200.0.0.1, nw_dst=200.0.0.254,
  tp_src=5001, tp_dst=49152 actions=mod_dl_src:00:00:00:bb:bb:bb, mod_dl_dst
  :00:00:00:00:00:02, mod_nw_dst:192.168.1.2, mod_tp_dst:40202, output:"s1-eth2"
```

Listing 3: Tabla de flujos OpenFlow en OVS durante la prueba de concurrencia TCP

7.3.2. Caso B: Concurrencia con UDP

Se replicó el entorno concurrente ahora con tráfico UDP de forma paralela para comprobar la coexistencia de reglas:

```
1 # En la xterm de h1 (Servidor):
2 iperf -s -u
3
4 # En la xterm de h2 (Cliente 1):
5 iperf -c 200.0.0.1 -u -t 20
6
7 # En la xterm de h3 (Cliente 2):
8 iperf -c 200.0.0.1 -u -t 20
```

Mientras ambas transmisiones se ejecutaban en simultáneo, se consultó el estado de las reglas activas mediante el comando `sudo ovs-ofctl dump-flows s1`.


```
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2# iperf -c 200.0.0.1 -u -t 20
Client connecting to 200.0.0.1, UDP port 5001
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 192.168.1.3 port 52103 connected with 200.0.0.1 port 5001
[ ID] Interval      Transfer     Bandwidth
[ 1] 0.0000-20.0192 sec  2.51 MBytes  1.05 Mbits/sec
[ 1] Sent 1708 datagrams
[ 3] WARNING: did not receive ack of last datagram after 10 tries.
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2#
```

Figura 17: Cliente h2: Consola iperf.

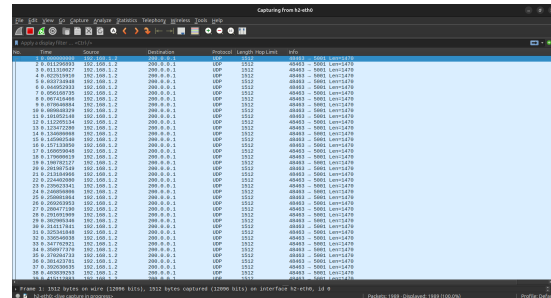


Figura 18: Cliente h2: Captura h2-eth0.

```
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2# iperf -c 200.0.0.1 -u -t 20
Client connecting to 200.0.0.1, UDP port 5001
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 192.168.1.2 port 48463 connected with 200.0.0.1 port 5001
[ ID] Interval      Transfer     Bandwidth
[ 1] 0.0000-20.0192 sec  2.51 MBytes  1.05 Mbits/sec
[ 1] Sent 1708 datagrams
[ 3] WARNING: did not receive ack of last datagram after 10 tries.
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2#
```

Figura 19: Cliente h3: Consola iperf.

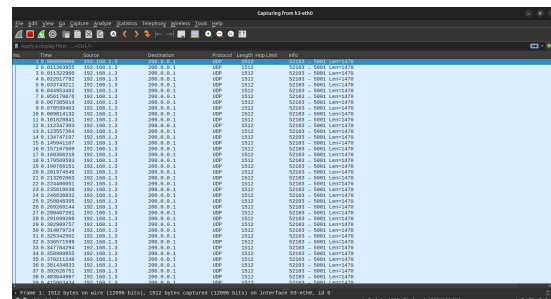


Figura 20: Cliente h3: Captura h3-eth0.

```
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2# iperf -s -u
Server listening on UDP port 5001
UDP buffer size: 208 KByte (default)
-----
[ 1] local 200.0.0.1 port 5001 connected with 200.0.0.254 port 52103
[ 2] local 200.0.0.1 port 5001 connected with 200.0.0.254 port 49152
[ ID] Interval      Transfer     Bandwidth      Jitter  Lost/Total Datagrams
[ 1] 0.0000-20.0078 sec  2.51 MBytes  1.05 Mbits/sec  0.054 ms 0/1787 (0%)
[ 1] 0.0000-20.0078 sec  1 datagrams received out-of-order
[ 3] WARNING: ack of last datagram failed.
[ 2] 0.0000-20.0180 sec  2.51 MBytes  1.05 Mbits/sec  0.008 ms 0/1787 (0%)
[ ID] Interval      Transfer     Bandwidth
[SUM] 0.0000-20.4411 sec  5.01 MBytes  2.06 Mbits/sec  0/3574 (0%)
[SUM] 0.0000-20.4411 sec  1 datagrams received out-of-order
[ 4] WARNING: ack of last datagram failed.
root@flor-PC:/home/flor/Documents/TPs/Redes/tp2#
```

Figura 21: Servidor h1 (UDP): Consola.

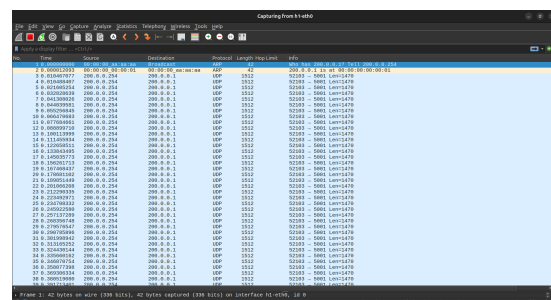


Figura 22: Servidor h1: Captura h1-eth0.

Resultados y Verificaciones en Concurrency UDP:

- **Salida de iperf UDP:** El servidor h1 recibió con éxito ambas transmisiones en hilos paralelos identificando las fuentes como 200.0.0.254:52103 y 200.0.0.254:49152 respectivamente. Esto demuestra el funcionamiento correcto del NAT dinámico sobre datagramas no orientados a conexión.
- **Análisis de capturas en Wireshark:** Al contrastar las capturas de los clientes con la del servidor, se observa la modificación de los encabezados IP y UDP realizada por el switch. En los clientes, se verifica que el tráfico origina desde sus direcciones IP privadas hacia la IP pública del servidor. En el servidor, la captura confirma que NAT es transparente, ya que el

tráfico llega desde la IP pública del router (200.0.0.254).

- **Análisis de los flujos OpenFlow instalados:** El resultado por terminal del switch s1 constató la coexistencia de las reglas dinámicas simétricas para UDP. Los contadores confirman que los flujos salientes procesaron los datagramas (n_packets=218 para el puerto 52103 y n_packets=180 para el puerto 49152):

```

1 # Flujos asociados a h3 (192.168.1.3)
2 cookie=0x0, duration=2.574s, table=0, n_packets=218, n_bytes=329616, idle_timeout
  =10, udp, in_port="s1-eth3", nw_src=192.168.1.3, nw_dst=200.0.0.1, tp_src
  =52103, tp_dst=5001 actions=mod_dl_src:00:00:00:aa:aa:aa, mod_dl_dst
  :00:00:00:00:00:01, mod_nw_src:200.0.0.254, mod_tp_src:52103, output:"s1-eth1"
3
4 cookie=0x0, duration=2.533s, table=0, n_packets=0, n_bytes=0, idle_timeout=10, udp,
  in_port="s1-eth1", nw_src=200.0.0.1, nw_dst=200.0.0.254, tp_src=5001, tp_dst
  =52103 actions=mod_dl_src:00:00:00:bb:bb:bb, mod_dl_dst:00:00:00:00:00:03,
  mod_nw_dst:192.168.1.3, mod_tp_dst:52103, output:"s1-eth3"
5
6 # Flujos asociados a h2 (192.168.1.2)
7 cookie=0x0, duration=2.141s, table=0, n_packets=180, n_bytes=272160, idle_timeout
  =10, udp, in_port="s1-eth2", nw_src=192.168.1.2, nw_dst=200.0.0.1, tp_src
  =48463, tp_dst=5001 actions=mod_dl_src:00:00:00:aa:aa:aa, mod_dl_dst
  :00:00:00:00:00:01, mod_nw_src:200.0.0.254, mod_tp_src:49152, output:"s1-eth1"
8
9 cookie=0x0, duration=2.141s, table=0, n_packets=0, n_bytes=0, idle_timeout=10, udp,
  in_port="s1-eth1", nw_src=200.0.0.1, nw_dst=200.0.0.254, tp_src=5001, tp_dst
  =49152 actions=mod_dl_src:00:00:00:bb:bb:bb, mod_dl_dst:00:00:00:00:00:02,
  mod_nw_dst:192.168.1.2, mod_tp_dst:48463, output:"s1-eth2"

```

Listing 4: Tabla de flujos OpenFlow durante la prueba de concurrencia UDP

7.4. Validación del comportamiento mediante tcpdump

También se utilizaron capturas mediante la herramienta `tcpdump` ejecutadas en segundo plano dentro de las terminales de los hosts.

```

1 # En la xterm del Servidor h1:
2 tcpdump -i h1-eth0 -n tcp port 5001 or arp
3
4 # En la xterm del Cliente h2:
5 tcpdump -i h2-eth0 -n tcp or arp

```

7.4.1. Prueba de validación con tráfico HTTP (TCPDump)

Con el propósito de validar la transparencia del NAT ante protocolos de capa de aplicación, se realizó una prueba utilizando un servidor web simple. Esta herramienta permite confirmar que los segmentos TCP son reescritos correctamente y que la resolución de nombres/direcciones no interfiere con el flujo de datos.

Metodología de captura: Se levantó un servidor web en el host h1 y se realizó una petición desde el host h3 hacia la IP pública 200.0.0.1.

```

1 # En la xterm del Servidor h1:
2 h1 python3 -m http.server 80 &
3 h1 tcpdump -i h1-eth0 -n -e -v &
4
5 # En la xterm del Cliente h3:
6 h3 curl http://200.0.0.1

```

Análisis de resultados: La ejecución de `tcpdump` permite capturar el tráfico a nivel de enlace y red. Al analizar la salida (ver imagen inferior), se observa cómo el paquete llega al servidor con la IP pública del router NAT (200.0.0.254) como origen, confirmando que la traducción de direcciones y puertos (PAT) se aplicó exitosamente antes de que el tráfico abandonara la red privada.

```
INFO:logger:MATCH: 192.168.1.3 pertenece a la red privada 192.168.1.0/24
INFO:logger:NAT SALIENTE: 192.168.1.3:48816 → 200.0.0.254:49152
INFO:logger:ENVIANDO: 200.0.0.254 → 200.0.0.1 | MAC: 00:00:00:aa:aa:aa → 00:00:00:00:00:01 | Out Port: 1
INFO:logger:ARP aprendido: 200.0.0.1 -> 00:00:00:00:00:01
INFO:logger:ARP Reply: 200.0.0.254 está en 00:00:00:aa:aa:aa → enviado a 200.0.0.1
INFO:logger:ARP aprendido: 192.168.1.3 -> 00:00:00:00:00:03
INFO:logger:ARP Reply: 192.168.1.254 está en 00:00:00:bb:bb:bb → enviado a 192.168.1.3
INFO:logger:LIMPIEZA NAT: Puerto público 49152 liberado.
```

Figura 23: Captura de tráfico HTTP mediante `tcpdump` en el servidor h1.

8. Conclusión

Este trabajo permitió reforzar conceptos de Redes Definidas por Software (SDN), confirmando la flexibilidad que ofrece separar el plano de control del de datos. Al abstraer funcionalidades de bajo nivel (como NAT y ARP) mediante POX, se demostró que no es necesario modificar hardware para implementar reglas lógicas complejas en el borde de una red.